



Scripted — AI Meeting Assistant

Technical report · Inter IIT Tech Meet, ML Bootcamp problem statement

Repository: github.com/PIYUSH-bit1818/inter-iit-meeting-assistant · October 2026

1. Summary

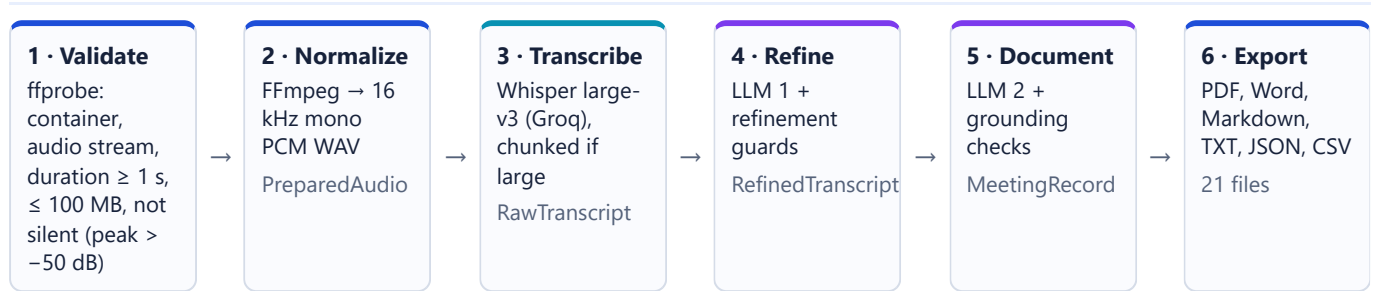
Scripted turns a recorded English meeting into a **raw transcript**, a **refined transcript**, **meeting minutes**, **key decisions** and **action items**. A FastAPI backend runs six stages: audio validation and normalization (FFmpeg), speech-to-text (**Whisper large-v3** on Groq), transcript refinement (**LLM 1**, Gemini), meeting documentation (**LLM 2**, a different Gemini model) and export. A Streamlit interface uploads the recording, shows live progress and presents and downloads every result. Deterministic code checks both language models: a refinement that changes meaning is rejected, and a decision or action item that is not supported by a verbatim transcript quote is withheld instead of shown.

2. Models and their roles

Stage	Model	Role	Input → output
Speech-to-text	whisper-large-v3 Groq API	Transcribes the normalized audio in English with segment timestamps (<code>verbose_json</code>). Recordings above the 25 MB request limit are split into ≤ 10 -minute windows with 10 s overlap; each segment is kept by the window whose midpoint owns it, so nothing is duplicated or lost.	16 kHz mono WAV → raw transcript : segments with id, start, end, text
LLM 1 Transcript refinement	<code>gemini-3.6-flash</code> , thinking level <i>low</i> Google Gemini API; optional fallback models (e.g. <code>gemini-3.8-flash</code> , <code>gemini-3.1-flash-lite</code>) on 429/503	Corrects recognition errors, domain terminology, punctuation and formatting, segment by segment. It may not add, remove, summarize or reinterpret content. Batches of $\leq 1,500$ words, with the full transcript as terminology context; structured JSON output.	Raw transcript → per-segment proposals → refined transcript (same ids and timestamps)
LLM 2 Meeting documentation	<code>gemini-3-flash-preview</code> , thinking level <i>medium</i> fallback <code>gemini-3.5-flash-lite</code>	Writes the summary, minutes by topic, decisions, proposals and action items. Every item cites evidence segment ids and a verbatim quote; owner and deadline are <code>null</code> unless stated. JSON schema enforced with <code>response_json_schema</code> .	Refined transcript with segment ids → meeting record

The two language models do different jobs. Refinement is a narrow, high-volume editing task, so it uses a fast Flash model with low thinking. Documentation has to reason across the whole meeting (agreement versus proposal, who owns a task, whether a deadline was stated), so it uses a different model with more thinking. Model IDs were taken from the live Gemini model list for our free-tier key rather than guessed, and every model is configurable in `.env` (`STT_MODEL`, `REFINE_MODEL`, `DOCUMENT_MODEL`, `*_FALLBACK_MODELS`). The models that actually served a job are recorded in its output and shown in the UI.

3. Pipeline and data flow



- **Jobs.** `POST /jobs` stores the upload and runs the stages in a background thread. Each stage records its status, start and finish time, message and error code. The UI polls `GET /jobs/{id}` every second and shows the real stage states, an elapsed clock and a rough time-left estimate.
- **Typed hand-offs.** Stages pass strict Pydantic objects (unknown fields rejected): `PreparedAudio` → `RawTranscript` → `RefinedTranscript` → `MeetingRecord`. The raw transcript is never modified; the refined transcript keeps the same segment ids and timestamps and lists every accepted and rejected change.
- **Traceability.** LLM 2 reads the refined text, but its evidence refers to the same segment ids, so every decision and action item can be traced back to the raw speech-to-text output.
- **Exports.** All formats are rendered from the same objects (`GET /jobs/{id}/exports/{key}`), so the human-readable (PDF, Word, Markdown) and machine-readable (JSON, CSV) records always list the same decisions and tasks. The uploaded audio and intermediate files are deleted when the job ends.

4. Preventing hallucination

Refinement guards (per segment)

- Word-level diff against the raw text. Changes must be small, local replacements (≤ 6 words, ≤ 2 extra words per replacement) that resemble the original (similarity ≥ 0.5 ; ≥ 0.75 for proper nouns). The whole segment must stay ≥ 0.6 similar.
- Numbers (including spelled-out forms such as “forty k”), dates, money, negations and modal verbs must be preserved. No new words may be introduced; only filler words (“um”, “uh”) may be dropped.
- A failing edit is rejected: the segment keeps its raw text and the rejection is reported with its reason.

Documentation grounding checks (per item)

- The evidence quote must appear in the cited segments, and numbers, dates, money and names in the item must appear in the evidence.
- A **decision** needs an explicit, non-negated agreement cue (agreed, decided, approved, confirmed...); otherwise it is a proposal. An **action item** needs an assignment or commitment.
- **Owner** and **deadline** must be stated in the evidence. Otherwise they are `null`, shown as “Not specified”, and never guessed.
- Items that fail are withheld with their reasons (UI “Safety checks” and the complete record), never shown as facts.

5. Results on the shared demo recording

`samples/demo_meeting/` is a 180 s synthetic meeting between three fictional speakers, made with Windows text-to-speech (no real people, no copyrighted audio). It includes decisions, proposals, a question, action items with and without owner and deadline, technical terms, numbers, a date and money. The folder holds the outputs of a real run of the application:

Stage	Result
Speech-to-text	35 segments, 407 words, one request (whisper-large-v3)
Refinement	2 segments corrected — "Cuba Ernets" → " Kubernetes ", "CI-CD" → "CI/CD"; 32 unchanged; 1 edit rejected (it added the word "are"). This run was served by the fallback <code>gemini-3.8-flash</code> while <code>gemini-3.6-flash</code> was returning 503 errors.
Decisions (2)	Migrate the Kubernetes cluster to version 1.29 · No changes to the database schema this sprint
Proposals (5)	Kept separate from decisions, e.g. moving the reporting database to PostgreSQL, a 2 a.m. nightly backup
Action items (2)	Update the deployment documentation — owner Rahul , deadline Friday, October 10th · Send the updated cost report to the finance team — owner and deadline Not specified
Minutes	A summary and 4 topics; 0 items withheld
Processing time	95–120 s for this 3-minute recording, 197 s for a 7.6-minute one. Transcription takes most of the time, mainly the audio upload (≈ 0.24 s per second of audio on our connection); refinement 26–64 s, documentation 8–20 s, export 2–5 s.

6. Verification

- **348 automated tests** (pytest, providers mocked): audio validation and normalization, chunking and segment ownership, refinement guards, grounding checks, schemas, exports (PDF and Word text identical to the JSON record) and API and runner error paths.
- **Browser end-to-end test** (Playwright, headless Microsoft Edge): **33/33 checks** on a real run. It covers upload, live progress with elapsed time and estimate, the results sections and all 20 downloads, which open and match the UI. It also confirms "Not specified" for missing owners and deadlines, and that no secret or internal path appears in the page or files.
- **Error paths:** empty, corrupt, silent, too short, oversized and unsupported files, and provider failures (rate limit, timeout, authentication, invalid output). Each produces a plain-language message with a suggested next step and a stable error code.

7. Engineering notes and limitations

- **Secrets:** API keys are read from `.env` as secret values, never logged or shown in the UI, and `.env` is excluded from git.
- **Resilience:** Gemini calls retry and fall back to the next configured model on 429/503. A long provider outage can still fail a job, with a clear message.
- **No speaker diarization:** Whisper does not label speakers, so an owner is assigned only when a name is spoken in the evidence.
- **Scope:** English only. Jobs live in memory, so a restart clears them and the app runs as a single process. Uploading compressed audio instead of 16-bit WAV would shorten transcription.